

Operating Systems Lab Assignment: Concurrency with Semaphores, Critical Sections, and Monitor Locks

adam white

October 10, 2026

1 Introduction

This report covers the six programs from the concurrency lab. Each one uses semaphores, mutexes, or condition variables to keep threads from messing up shared data.

2 Exercise 1: Producer-Consumer in C

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t full, empty;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        int item = rand() % 100;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2);
    }
}
```

```

    }
    return NULL;
}

int main() {
    pthread_t prod, cons;
    sem_init(&full, 0, 0);
    sem_init(&empty, 0, BUFFER_SIZE);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&full);
    sem_destroy(&empty);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

Explanation: The empty semaphore starts at 5 and counts open spots in the buffer. The full semaphore starts at 0 and counts items in the buffer. The producer waits on empty before adding and posts full after. The consumer waits on full before taking and posts empty after. The mutex makes sure only one thread changes the buffer at a time.

Analysis: The producer sleeps 1 second and the consumer sleeps 2, so the producer gets ahead and the consumer finishes the last items at the end. Items came out in the same order they went in. If the semaphores were removed, the producer could overwrite items and the consumer could read an empty spot. Without the mutex, two threads could change the buffer at the same time.



```

@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $ ./producer_consumer
Produced: 83
Consumed: 83
Produced: 86
Consumed: 86
Produced: 77
Produced: 15
Consumed: 77
Produced: 93
Produced: 35
Consumed: 15
Produced: 86
Produced: 92
Consumed: 93
Produced: 49
Produced: 21
Consumed: 35
Consumed: 86
Consumed: 92
Consumed: 49
Consumed: 21
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $

```

Figure 1: Compilation and execution of producer_consumer.c

3 Exercise 2: Bank Account Simulation in C++

```
#include <iostream>
```

```

#include <thread>
#include <mutex>
#include <condition_variable>
#include <vector>
#include <chrono>

class BankAccount {
private:
    int balance;
    std::mutex mtx;
    std::condition_variable cv;
public:
    BankAccount(int initial_balance) : balance(initial_balance) {}

    void deposit(int amount) {
        std::unique_lock<std::mutex> lock(mtx);
        balance += amount;
        std::cout << "Deposited:␣" << amount << " ,␣New␣Balance:␣" << balance <<
            "\n";
        cv.notify_all();
    }

    void withdraw(int amount) {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [&] { return balance >= amount; });
        balance -= amount;
        std::cout << "Withdrew:␣" << amount << " ,␣New␣Balance:␣" << balance <<
            "\n";
    }
};

void user_transactions(BankAccount &account) {
    account.deposit(100);
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
    account.withdraw(50);
}

int main() {
    BankAccount account(100);
    std::vector<std::thread> users;
    for (int i = 0; i < 5; ++i) {
        users.push_back(std::thread(user_transactions, std::ref(account)));
    }
    for (auto &t : users) {
        t.join();
    }
    return 0;
}

```

Explanation: Deposit and withdraw both lock the mutex so only one thread changes the balance at a time. In withdraw, cv.wait makes the thread wait until the balance is at least the amount. Deposit calls notify_all to wake up any thread that is waiting.

Analysis: Five threads each deposit 100 and withdraw 50. The balance went up to 600 and ended at 350, which is correct. No thread had to wait because the balance was always high enough. Without the mutex, two threads could change the balance at once and one update could be lost. Without the condition variable, the balance could go negative.

4 Exercise 3: Reader-Writer Problem in C

```

#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>

```

```
Problems Output Debug Console Terminal Ports
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $ ./bank_account
Deposited: 100, New Balance: 200
Deposited: 100, New Balance: 300
Deposited: 100, New Balance: 400
Deposited: 100, New Balance: 500
Deposited: 100, New Balance: 600
Withdrew: 50, New Balance: 550
Withdrew: 50, New Balance: 500
Withdrew: 50, New Balance: 450
Withdrew: 50, New Balance: 400
Withdrew: 50, New Balance: 350
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $
```

Figure 2: Compilation and execution of bank_account.cpp

```
#include <unistd.h>

sem_t rw_mutex, mutex;
int reader_count = 0;
int shared_data = 0;

void *reader(void *arg) {
    sem_wait(&mutex);
    reader_count++;
    if (reader_count == 1) sem_wait(&rw_mutex);
    sem_post(&mutex);

    printf("Reader reads: %d\n", shared_data);
    sleep(1);

    sem_wait(&mutex);
    reader_count--;
    if (reader_count == 0) sem_post(&rw_mutex);
    sem_post(&mutex);
    return NULL;
}

void *writer(void *arg) {
    sem_wait(&rw_mutex);
    shared_data++;
    printf("Writer writes: %d\n", shared_data);
    sleep(1);
    sem_post(&rw_mutex);
    return NULL;
}

int main() {
    pthread_t readers[3], writers[2];
    sem_init(&rw_mutex, 0, 1);
    sem_init(&mutex, 0, 1);

    for (int i = 0; i < 3; i++) pthread_create(&readers[i], NULL, reader, NULL);
    for (int i = 0; i < 2; i++) pthread_create(&writers[i], NULL, writer, NULL);

    for (int i = 0; i < 3; i++) pthread_join(readers[i], NULL);
    for (int i = 0; i < 2; i++) pthread_join(writers[i], NULL);
}
```

```

    sem_destroy(&rw_mutex);
    sem_destroy(&mutex);
    return 0;
}

```

Explanation: The mutex semaphore protects reader_count. The first reader waits on rw_mutex to block writers, and the last reader posts it so writers can go. Writers wait on rw_mutex, so only one writer can use the data and no readers can be in at the same time.

Analysis: This is readers-first. All three readers read 0 together, then the writers wrote 1 and 2 one at a time. If readers kept coming, a writer might never get a turn. Without the semaphores, a reader could read while a writer is changing the data, and two writers could overwrite each other.

```

Problems  Output  Debug Console  Terminal  Ports
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $ ./reader_writer
Reader reads: 0
Reader reads: 0
Reader reads: 0
Writer writes: 1
Writer writes: 2
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $

```

Figure 3: Compilation and execution of reader_writer.c

5 Exercise 4: Dining Philosophers in C++

```

#include <iostream>
#include <thread>
#include <mutex>
#include <vector>
#include <chrono>

#define NUM_PHILOSOPHERS 5

std::mutex forks[NUM_PHILOSOPHERS];

void philosopher(int id) {
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;
    if (id == NUM_PHILOSOPHERS - 1) { // Avoid deadlock
        std::swap(left, right);
    }

    for (int i = 0; i < 3; ++i) {
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
        forks[left].lock();
        forks[right].lock();
        std::cout << "Philosopher " << id << " is eating.\n";
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
        forks[right].unlock();
        forks[left].unlock();
    }
}

int main() {
    std::vector<std::thread> philosophers;
    for (int i = 0; i < NUM_PHILOSOPHERS; ++i) {
        philosophers.push_back(std::thread(philosopher, i));
    }
}

```

```

    }
    for (auto &t : philosophers) {
        t.join();
    }
    return 0;
}

```

Explanation: Each fork is a mutex and a philosopher needs two forks to eat. If everyone picked up their left fork first, they could all wait forever. The last philosopher swaps the order, so everyone picks up the lower numbered fork first. This stops the circle of waiting, so there is no deadlock.

Analysis: Each philosopher ate 3 times for 15 lines total. In my run they went in the order 1, 3, 0, 2, 4 each round. Neighbors never ate at the same time since they share a fork. Without the swap, the program could freeze.

```

@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $ ./dining_philosophers
Philosopher 1 is eating.
Philosopher 3 is eating.
Philosopher 0 is eating.
Philosopher 2 is eating.
Philosopher 4 is eating.
Philosopher 1 is eating.
Philosopher 3 is eating.
Philosopher 0 is eating.
Philosopher 2 is eating.
Philosopher 4 is eating.
Philosopher 1 is eating.
Philosopher 3 is eating.
Philosopher 0 is eating.
Philosopher 2 is eating.
Philosopher 4 is eating.

```

Figure 4: Compilation and execution of dining_philosophers.cpp

6 Exercise 5: Sleeping Barber Problem in C

```

#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

#define NUM_CHAIRS 3
#define NUM_CUSTOMERS 5

sem_t customers, barber, chairs;
pthread_mutex_t mutex;

void *barber_thread(void *arg) {
    while (1) {
        sem_wait(&customers);
        pthread_mutex_lock(&mutex);
        sem_post(&chairs);
        pthread_mutex_unlock(&mutex);
        printf("Barber is cutting hair.\n");
        sleep(2);
        sem_post(&barber);
    }
    return NULL;
}

void *customer_thread(void *arg) {
    pthread_mutex_lock(&mutex);

```

```

    if (sem_trywait(&chairs) == 0) {
        sem_post(&customers);
        pthread_mutex_unlock(&mutex);
        sem_wait(&barber);
        printf("Customer got haircut.\n");
    } else {
        pthread_mutex_unlock(&mutex);
        printf("Customer left (no chairs).\n");
    }
    return NULL;
}

int main() {
    pthread_t barber_t, customers_t[NUM_CUSTOMERS];
    sem_init(&customers, 0, 0);
    sem_init(&barber, 0, 0);
    sem_init(&chairs, 0, NUM_CHAIRS);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&barber_t, NULL, barber_thread, NULL);

    for (int i = 0; i < NUM_CUSTOMERS; i++) {
        pthread_create(&customers_t[i], NULL, customer_thread, NULL);
        sleep(1);
    }

    for (int i = 0; i < NUM_CUSTOMERS; i++) {
        pthread_join(customers_t[i], NULL);
    }

    // Note: Barber thread runs indefinitely; in practice, add termination
    // logic.
    sem_destroy(&customers);
    sem_destroy(&barber);
    sem_destroy(&chairs);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

Explanation: The customers semaphore counts waiting customers, and the barber sleeps on it when no one is there. The chairs semaphore starts at 3 and counts open chairs. The barber semaphore tells a customer their haircut is done. A customer uses `sem_trywait` on chairs and leaves if none are open. If it gets a chair, it posts customers to wake the barber and waits on barber. The mutex keeps two customers from checking the chairs at the same time.

Analysis: Customers come every second and a haircut takes 2 seconds. With 3 chairs, all five customers get haircuts and nobody left. The barber thread never ends, but the program still exits when main returns. Without the chairs semaphore there would be no limit on waiting customers.

```

Problems  Output  Debug Console  Terminal  Ports

Barber is cutting hair.
Barber is cutting hair.
Customer got haircut.
Barber is cutting hair.
Customer got haircut.
Barber is cutting hair.
Customer got haircut.
Barber is cutting hair.
Customer got haircut.
Barber is cutting hair.
Customer got haircut.
Customer got haircut.
@whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $

```

Figure 5: Compilation and execution of `sleeping_barber.c`

7 Exercise 6: Bounded Buffer with Condition Variables in C++

```
#include <iostream>
#include <cstdlib>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <queue>
#include <chrono>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10

std::queue<int> buffer;
std::mutex mtx;
std::condition_variable not_full, not_empty;

void producer() {
    for (int i = 0; i < NUM_ITEMS; ++i) {
        std::unique_lock<std::mutex> lock(mtx);
        not_full.wait(lock, [] { return buffer.size() < BUFFER_SIZE; });
        int item = rand() % 100;
        buffer.push(item);
        std::cout << "Produced:␣" << item << "\n";
        not_empty.notify_one();
        lock.unlock();
        std::this_thread::sleep_for(std::chrono::milliseconds(1000));
    }
}

void consumer() {
    for (int i = 0; i < NUM_ITEMS; ++i) {
        std::unique_lock<std::mutex> lock(mtx);
        not_empty.wait(lock, [] { return !buffer.empty(); });
        int item = buffer.front();
        buffer.pop();
        std::cout << "Consumed:␣" << item << "\n";
        not_full.notify_one();
        lock.unlock();
        std::this_thread::sleep_for(std::chrono::milliseconds(2000));
    }
}

int main() {
    std::thread prod(producer);
    std::thread cons(consumer);

    prod.join();
    cons.join();
    return 0;
}
```

Explanation: The producer waits on not_full until the queue has room, adds an item, and notifies not_empty. The consumer waits on not_empty until there is an item, takes it, and notifies not_full. The wait call releases the lock while sleeping so the other thread can run.

Analysis: The output was the same as Exercise 1 since the timing is the same. In Exercise 1 the semaphores keep the counts. Here the condition variables check the actual queue size. This version is easier to read and uses one mutex, but a thread has to hold the lock to wait. Semaphores are simpler when you only need to count.

```
Problems  Output  Debug Console  Terminal  Ports
● @whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $ ./bounded_buffer
Produced: 83
Consumed: 83
Produced: 86
Consumed: 86
Produced: 77
Produced: 15
Consumed: 77
Produced: 93
Produced: 35
Consumed: 15
Produced: 86
Produced: 92
Consumed: 93
Produced: 49
Produced: 21
Consumed: 35
Consumed: 86
Consumed: 92
Consumed: 49
Consumed: 21
❖ @whiadam → /workspaces/cs375-labs/cs375-alilab5prod_consumer (main) $
```

Figure 6: Compilation and execution of bounded_buffer.cpp